

E04 — Initialization & Noise Robustness: Muon vs SGD

Overview

This experiment evaluates how **initialization scale** and **observation noise** affect the convergence of Muon and SGD on the matrix sensing problem. Robustness to initialization and noise are critical practical concerns: in real applications, good initializations may not be available, and measurements always contain noise.

Why this matters: Understanding how algorithms behave under varying initialization scales reveals their convergence basin properties. Testing with observation noise evaluates practical applicability where measurements are never perfect.

Experiment ID: E04 | **Problem Domain:** Matrix Sensing | **Algorithms:** Muon-Exact vs SGD

Scientific Question

Hypothesis

Muon's spectral normalization provides more consistent convergence across different initialization scales, as the normalized update direction is less sensitive to the initial gradient magnitude. Under observation noise, Muon's spectral truncation may also provide implicit regularization.

Key Metrics

- ϵ : Iterations to reach $\epsilon = 0.01$
- **min_loss**: Best loss achieved
- **final_loss**: Loss after 2000 iterations
- **time_s**: Wall-clock time

Statistical Framework

Paired t-tests per (init_scale, noise) combination with Cohen's d effect size.

Experimental Design

Parameter	Value	Description
d	50	Matrix dimension
r	5	Target rank
m	dr = 500	Measurements
lr	0.01	Learning rate
init_scale	{0.001, 0.01, 0.1, 1.0}	Initialization std dev
noise	{0, 0.01, 0.1}	Observation noise levels
dist	normal	Gaussian matrices
spectrum	hard-cutoff	Flat top-r singular values
Iterations	2000	Max iterations
Seeds	5	Replications per config
ϵ	0.01	Convergence threshold

Total runs: 2 algos x 4 init_scales x 3 noises x 5 seeds = 120 runs

Data generation: Each measurement is $y_i = \langle \mathbf{z}_i, \mathbf{x}^* \rangle + \epsilon_i$ where $\mathbf{z}_i \sim (0, \frac{2}{n})$ and $\epsilon_i \in \{0, 0.01, 0.1\}$. Initialization: $\mathbf{z}^{(0)} \sim (0, \frac{2}{n_{init}})$.

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
plt.style.use('seaborn-v0_8-whitegrid')
df = pd.read_csv('../results_v3/E04_detailed_results.csv')
print(f"Shape: {df.shape}")
print(f"Algorithms: {df['algo'].unique()}")
print(f"Init scales: {sorted(df['init_scale'].unique())}")
print(f"Noise levels: {sorted(df['noise'].unique())}")
df.head()
```

Exploratory Data Analysis

```
In [ ]: summary = df.groupby(['algo', 'init_scale', 'noise'])['K_epsilon'].agg(['mean'])
print("K_epsilon summary by (algo, init_scale, noise):")
print(summary)

# Convergence summary
print("Convergence rates:")
conv = df.groupby(['algo', 'init_scale', 'noise'])['I_conv'].mean()
print(conv)
```

Comparative Analysis: Paired t-tests per (init_scale, noise)

```
In [ ]: print("Paired t-tests: Muon vs SGD per (init_scale, noise)")
print("-" * 80)
results = []
for init in sorted(df['init_scale'].unique()):
    for noise in sorted(df['noise'].unique()):
        muon_k = df[(df['algo']=='Muon-Exact') & (df['init_scale']==init) & (df['noise']==noise)]
        sgd_k = df[(df['algo']=='SGD') & (df['init_scale']==init) & (df['noise']==noise)]
        if len(muon_k) > 1 and len(muon_k) == len(sgd_k):
            t_stat, p_val = stats.ttest_rel(muon_k, sgd_k)
            diff = muon_k - sgd_k
            cohens_d = diff.mean()/diff.std(ddof=1) if diff.std(ddof=1)>0 else 0
            sig = "***" if p_val < 0.001 else "*" if p_val < 0.01 else ""
            print(f"init={init:.4f} noise={noise:.2f}: t={t_stat:>+6.3f} p={p_val:>+6.3f} sig={sig}")
            results.append({'init_scale': init, 'noise': noise, 'muon_mean': muon_k.mean(),
                           'sgd_mean': sgd_k.mean(), 't_stat': t_stat, 'p_val': p_val, 'cohens_d': cohens_d})

import pandas as pd
print(
    "\n", pd.DataFrame(results).to_string())
```

Visualization 1: K_ϵ vs Initialization Scale (Grouped by Noise)

```
In [ ]: fig, axes = plt.subplots(1, 3, figsize=(16, 5))
noises = sorted(df['noise'].unique())
init_scales = sorted(df['init_scale'].unique())

for idx, noise in enumerate(noises):
    ax = axes[idx]
    for algo, color in [('Muon-Exact', '#2E86AB'), ('SGD', '#F18F01')]:
        subset = df[(df['algo']==algo) & (df['noise']==noise)]
        means = [subset[subset['init_scale']==s]['K_epsilon'].mean() for s in init_scales]
        stds = [subset[subset['init_scale']==s]['K_epsilon'].std() for s in init_scales]
        ax.errorbar(init_scales, means, yerr=stds, label=algo, color=color,
                    linestyle='solid', marker='o')
    ax.set_xlabel('Initialization Scale', fontsize=11)
    ax.set_ylabel('Average K_epsilon', fontsize=11)
    ax.set_title(f'Noise = {noise}', fontsize=12, fontweight='bold')
    ax.set_xscale('log')
    ax.legend(); ax.grid(alpha=0.3)

fig.suptitle('E04: Convergence Speed vs Initialization Scale', fontsize=15,
            fontweight='bold')
plt.tight_layout(); plt.show()
```

Visualization 2: K_ϵ vs Noise Level (Grouped by Init Scale)

```
In [ ]: fig, axes = plt.subplots(1, 4, figsize=(18, 4.5))
        for idx, init in enumerate(init_scales):
            ax = axes[idx]
            for algo, color in [('Muon-Exact', '#2E86AB'), ('SGD', '#F18F01')]:
                subset = df[(df['algo']==algo)&(df['init_scale']==init)]
                means = [subset[subset['noise']==n]['K_epsilon'].mean() for n in noises]
                stds = [subset[subset['noise']==n]['K_epsilon'].std() for n in noises]
                ax.errorbar(noises, means, yerr=stds, label=algo, color=color, marker='o')
            ax.set_xlabel('Noise Level', fontsize=11)
            ax.set_ylabel('Average K_epsilon', fontsize=11)
            ax.set_title(f'Init = {init}', fontsize=12, fontweight='bold')
            ax.legend(); ax.grid(alpha=0.3)
        fig.suptitle('E04: Convergence Speed vs Observation Noise', fontsize=15, fontweight='bold')
        plt.tight_layout(); plt.show()
```

Visualization 3: Final Loss Comparison

```
In [ ]: fig, axes = plt.subplots(1, 3, figsize=(16, 5))
        for idx, noise in enumerate(noises):
            ax = axes[idx]
            for algo, color in [('Muon-Exact', '#2E86AB'), ('SGD', '#F18F01')]:
                subset = df[(df['algo']==algo)&(df['noise']==noise)]
                means = [subset[subset['init_scale']==s]['final_loss'].mean() for s in init_scales]
                stds = [subset[subset['init_scale']==s]['final_loss'].std() for s in init_scales]
                ax.errorbar(init_scales, means, yerr=stds, label=algo, color=color, marker='o')
            ax.set_xlabel('Initialization Scale', fontsize=11)
            ax.set_ylabel('Final Loss (log scale)', fontsize=11)
            ax.set_title(f'Noise = {noise}', fontsize=12, fontweight='bold')
            ax.set_xscale('log'); ax.set_yscale('log')
            ax.legend(); ax.grid(alpha=0.3)
        fig.suptitle('E04: Final Loss vs Initialization Scale', fontsize=15, fontweight='bold')
        plt.tight_layout(); plt.show()
```

Detailed Statistical Tests

```

In [ ]: print("=" * 80)
print("Detailed Tests by Initialization Scale and Noise")
print("=" * 80)
for init in sorted(df['init_scale'].unique()):
    for noise in sorted(df['noise'].unique()):
        muon_d = df[(df['algo']=='Muon-Exact') & (df['init_scale']==init) & (df['noise']==noise)]
        sgd_d = df[(df['algo']=='SGD') & (df['init_scale']==init) & (df['noise']==noise)]
        if len(muon_d) == 0 or len(sgd_d) == 0: continue
        muon_k, sgd_k = muon_d['K_epsilon'].values, sgd_d['K_epsilon'].values
        if len(muon_k) != len(sgd_k): continue
        diff = muon_k - sgd_k
        t_k, p_k = stats.ttest_rel(muon_k, sgd_k)
        d_k = diff.mean()/diff.std(ddof=1) if diff.std(ddof=1)>0 else 0
        print(f"init={init:.4f} noise={noise:.2f}:")
        print(f"    K_eps: Muon {muon_k.mean():.1f}+/-{muon_k.std():.1f}   SGD {sgd_k.mean():.1f}+/-{sgd_k.std():.1f}")
        print(f"    t={t_k:+.4f}, p={p_k:.6f}, Cohen's d={d_k:+.4f}")

```

Conclusions & Interpretation

Key Findings

1. Initialization Scale Effect:

- **Small initialization** (init_scale = 0.001): Both algorithms converge similarly, with slight Muon advantage.
- **Medium initialization** (init_scale = 0.01): Optimal for both algorithms; convergence is fastest.
- **Large initialization** (init_scale = 0.1, 1.0): Both algorithms require more iterations, with Muon showing larger degradation at init_scale = 1.0.

2. Noise Robustness:

- **Zero noise**: SGD achieves lower final loss; Muon converges in fewer iterations.
- **Moderate noise** (noise = 0.01): Both algorithms adapt well, with minimal impact on convergence.
- **High noise** (noise = 0.1): Both algorithms show increased ϵ , but the relative performance pattern remains stable.

3. Interaction Effects:

- The combination of large initialization and high noise creates the most challenging scenario for both algorithms.
- SGD shows more consistent performance across the full parameter grid.

4. Practical Implications:

- **Use init_scale ~ 0.01** for best convergence with both algorithms.
- Muon's spectral normalization does not provide special robustness to initialization scale or noise in this problem setting.

- SGD remains the more robust choice when initialization quality or noise level is uncertain.